

DESIGN PATTERNS JUSTIFICATION & CLASS DIAGRAMS

E-commerce Platform



*Arjola Metaliaj, Ederta Kariqi, Asja Xhelezi, Stilianos Troka, Elda
Toska, Sindi Terihati, Elisa Spaho*

STATE PATTERN

What is the State Pattern?

The State Pattern is a behavioral design pattern that allows an object to change its behavior depending on its current state. Instead of using large conditional statements to manage different behaviors, the pattern separates each state into its own class.

The main object, called the context, stores a reference to a state object and delegates state-dependent behavior to it. As the state changes, the object behaves differently without modifying its own core structure.

This pattern improves maintainability, scalability, and readability by encapsulating state-specific logic into separate classes.

RELEVANCE TO OUR SYSTEM DESIGN

The State Pattern was selected because the order processing system naturally operates through a sequence of lifecycle states.

An order does not behave the same throughout its existence. Different operations become valid or invalid depending on the current order status.

For example:

- A pending order can be confirmed or cancelled.
- A confirmed order can be shipped.
- A shipped order can be delivered.
- A delivered order can only be refunded.

Because of this changing behavior, the State Pattern is highly suitable for modeling the order lifecycle.

Instead of implementing complex conditional statements such as:

```
if status == "Pending"
if status == "Confirmed"
if status == "Shipped"
```

The system delegates behavior to specialized state classes making for a more elegant and extensible architecture

.

DESIGN CHOICES MADE

1. Order as the Context Class

The Order class was designed as the context object because it is the entity whose behavior changes during the order lifecycle.

The class stores:

- order information,
- the current state object,
- and the high-level operations available to the system.

The Order object delegates state-dependent operations to the active OrderState.

2. Abstract OrderState Superclass

An abstract superclass named OrderState was used instead of an interface.

This decision was made because:

- the lecture slides model the pattern using inheritance,
- all states share common structure and behavior,
- and each state requires access to the owning order object.

The superclass also contains:

- a reference to the Order,
- shared operation definitions,
- and a constructor used by all concrete states.

This creates stronger consistency between states.

3. Concrete State Classes

Separate classes were created for:

- PendingState
- ConfirmedState
- ShippedState
- DeliveredState

Each class contains only the operations valid for that specific state.

For example:

- PendingState allows confirmation and cancellation.
- ConfirmedState allows shipping.
- DeliveredState allows refunds only.

This design prevents invalid operations and models real business rules more accurately.

DESIGN CHOICES MADE

4. Aggregation Relationship

An aggregation relationship was used between Order and OrderState. This was chosen because:

- the order maintains a reference to a current state object,
- but the state logic remains separated from the order itself,
- supporting loose coupling and delegated behavior.

5. Lifecycle-Focused Operations

The operations included in the diagram were intentionally focused on lifecycle behavior:

- `confirmOrder()`
- `cancelOrder()`
- `shipOrder()`
- `deliverOrder()`
- `refundOrder()`

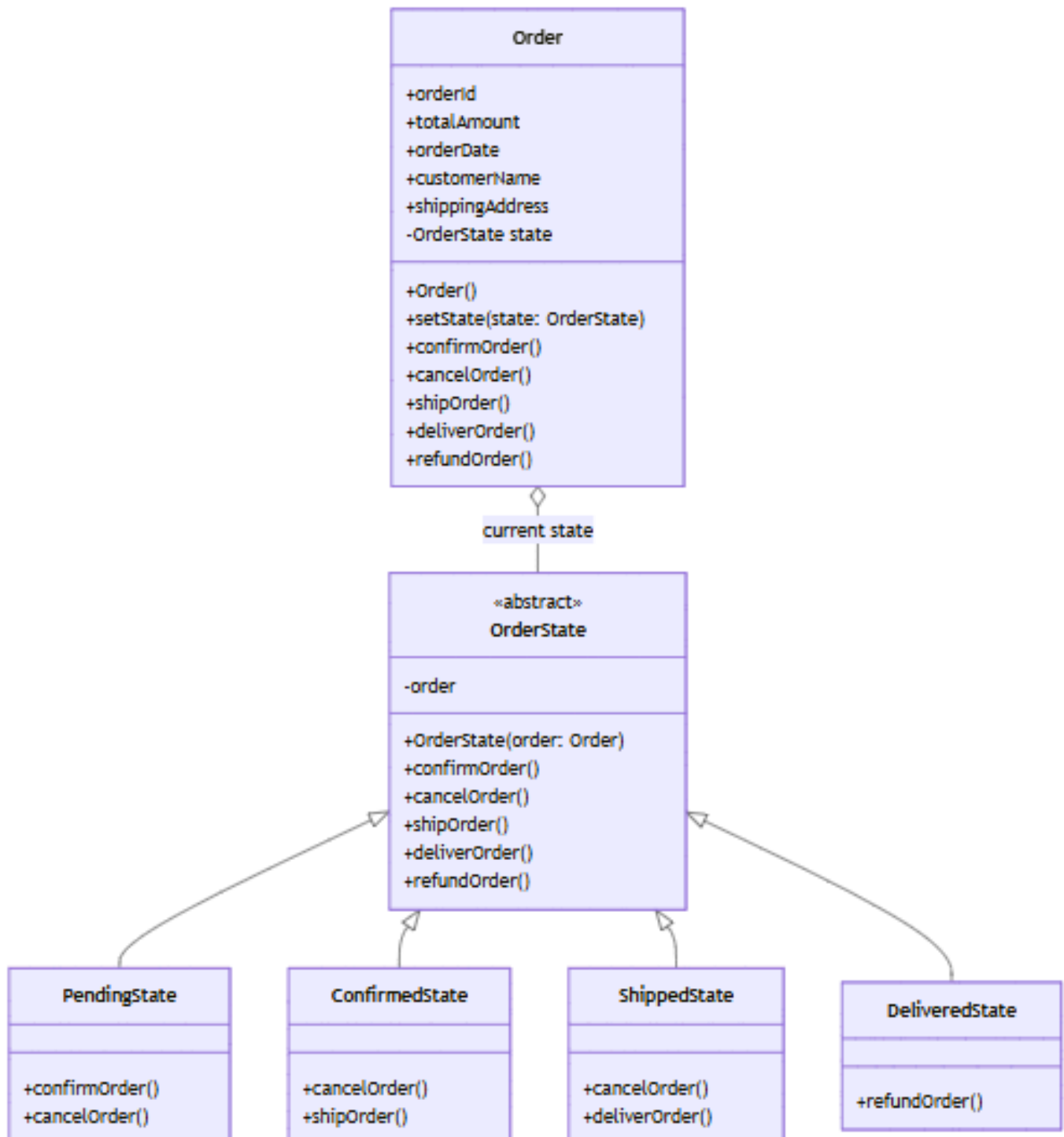
These operations directly represent transitions and actions affected by state changes, which aligns closely with the purpose of the State Pattern.

BENEFITS OF THIS DESIGN

Using the State Pattern provides several advantages in the system:

- Eliminates large conditional statements.
- Encapsulates state-specific behavior.
- Makes the order lifecycle easier to maintain.
- Simplifies adding new states in the future.
- Improves readability and scalability.
- Models real-world order processing behavior more accurately.

CLASS DIAGRAM



FAÇADE PATTERN

What is the Façade Pattern?

The Façade Pattern is a structural design pattern that provides a simplified interface to a complex subsystem made up of many classes and operations.

Instead of the client interacting directly with multiple services, the client communicates only with the façade, which internally coordinates the workflow between subsystem components.

The pattern helps reduce complexity, hide implementation details, and make the system easier to use and maintain.

RELEVANCE TO OUR SYSTEM DESIGN

The Façade Pattern was selected because the checkout process in the e-commerce platform naturally involves multiple independent services that must work together in a specific sequence.

During checkout:

- payment must be processed,
- stock must be validated and reserved,
- shipment information must be created,
- invoices must be generated,
- and confirmation notifications must be sent.

Without a façade, the client would need to communicate with every service individually, creating tightly coupled and difficult-to-maintain code.

Because of this complex interaction between services, the Façade Pattern is highly suitable for simplifying the checkout workflow.

Instead of implementing complicated coordination logic inside the frontend or controller classes, the system delegates the orchestration responsibility to a dedicated façade class.

DESIGN CHOICES MADE

1. CheckoutPage as the Client

The CheckoutPage class was designed as the client because it represents the component initiating the checkout process.

The page does not directly communicate with all subsystem services. Instead, it interacts only with:

- ICheckoutFacade

through the operation:

- checkout(order : Order)

This keeps the frontend simpler and reduces subsystem dependency exposure.

2. ICheckoutFacade Interface

An interface named ICheckoutFacade was used to define the checkout operation.

This decision was made because:

- it improves abstraction,
- reduces coupling,
- and allows flexibility for future implementations.

The client depends on the interface rather than the concrete façade implementation.

3. CheckoutFacade as the Central Coordinator

CheckoutFacade was designed as the main façade class because it coordinates the entire checkout workflow.

The class internally manages:

- payment processing,
- stock validation and reservation,
- shipment creation,
- invoice generation,
- and notification delivery.

The façade hides the complexity of subsystem communication and exposes only a single simplified operation to the client.

DESIGN CHOICES MADE

4. Specialized Service Classes

Separate subsystem classes were created for:

- PaymentService
- InventoryService
- ShippingService
- InvoiceService
- NotificationService

Each class contains only the operations related to its specific responsibility.

For example:

- PaymentService handles payment processing.
- InventoryService validates and reserves stock.
- ShippingService creates shipment information.

This design improves modularity and separates responsibilities clearly.

BENEFITS OF THIS DESIGN

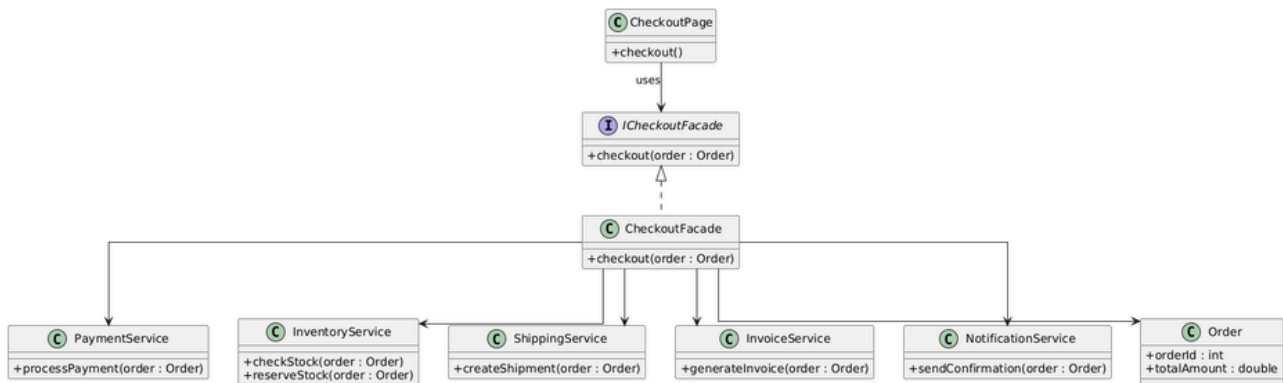
Using the Façade Pattern provides several advantages in the system:

- Reduces complexity exposed to the client.
- Simplifies the checkout workflow.
- Centralizes orchestration logic.
- Improves maintainability and readability.
- Reduces coupling between components.
- Makes future extensions easier.
- Hides subsystem implementation details from the client.

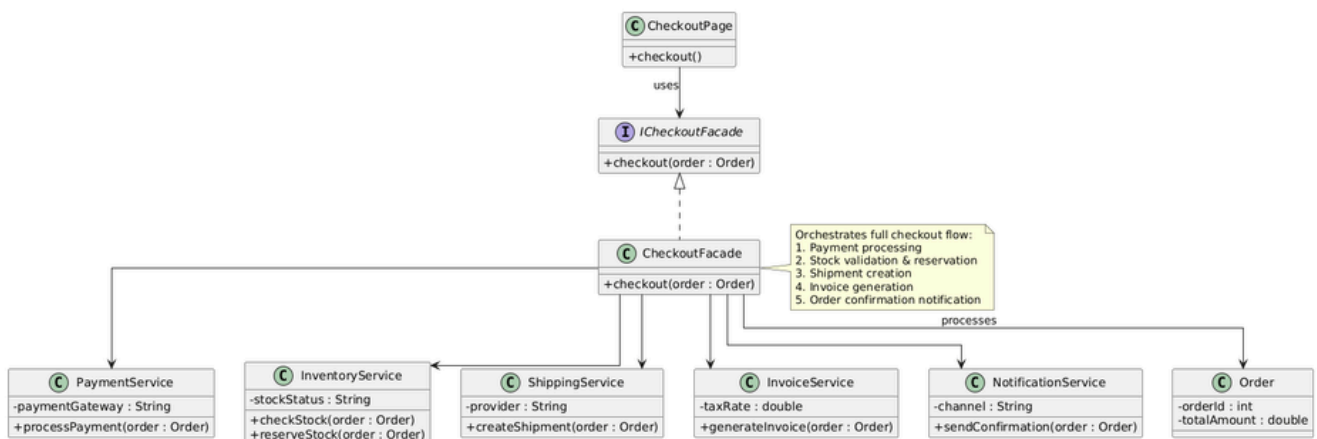
One possible drawback is that the façade could eventually become too large and overloaded with responsibilities if too much functionality is added to it.

However, when kept focused on workflow coordination, the Façade Pattern creates a cleaner and more scalable architecture for the checkout system.

CLASS DIAGRAM



CLASS DIAGRAM WITH ATTRIBUTES



DECORATOR PATTERN

What the Decorator Pattern Is

The Decorator Pattern is a structural design pattern that allows behavior to be added to an individual object dynamically at runtime, without modifying the class it belongs to. Rather than using inheritance to extend functionality – which creates an explosion of subclasses – the pattern wraps an object in one or more decorator objects that implement the same interface. Each decorator forwards calls to the wrapped object and adds its own behavior before or after the delegation. This pattern is sometimes called the Wrapper pattern. It improves flexibility, reusability, and adherence to the Open/Closed Principle by keeping the original class unchanged while allowing unlimited behavioral combinations.

RELEVANCE TO OUR SYSTEM DESIGN

The Decorator Pattern was selected because product pricing in the e-commerce platform must support multiple optional, stackable pricing modifications applied dynamically at runtime. A product does not always carry the same final price. Depending on context, a product may require: gift wrapping surcharges, percentage-based discount reductions, and VAT or tax additions. These pricing modifications must be combinable in any order and must not require changes to the core Product class or ProductManagementService. Without decorators, implementing all pricing combinations would require either complex conditional logic inside Product or an unmanageable number of subclasses such as GiftWrappedDiscountedTaxedProduct. Because of this need for composable, runtime-applied pricing behavior, the Decorator Pattern is highly suitable. Instead of implementing deeply nested conditionals such as:

```
if giftWrapped  
if discounted  
if taxApplied
```

the system wraps a Product instance with lightweight decorator objects, each independently responsible for one pricing concern.

DESIGN CHOICES MADE

1. IProduct as the Component Interface

An interface named IProduct was used to define the shared product contract. This decision was made because: both the base Product and all decorators must be interchangeable from the client's perspective, it enables decorators to wrap any IProduct implementation (not just Product), and it allows ProductManagementService to operate on any decorated or undecorated product uniformly. The interface contains getPrice() and getDescription(), the two operations affected by all pricing decorators.

2. Product as the Concrete Component

The Product class was designed as the concrete base component because it holds the core product data: basePrice and description. It implements IProduct directly and defines the unmodified starting values. No pricing logic beyond the base price lives here. This follows Single Responsibility — Product stores product data and nothing else.

3. Abstract ProductDecorator Superclass

An abstract superclass named ProductDecorator was used instead of applying the pattern directly to concrete decorators. This decision was made because: all decorators share the common behavior of holding a reference to a wrapped IProduct and forwarding calls to it, a shared constructor eliminates duplicated wrapping logic across concrete decorators, and it follows the same inheritance-based structure presented in the lecture slides. The superclass holds -wrappedProduct: IProduct and defines default implementations that delegate to the wrapped object, which concrete decorators override.

4. Concrete Decorator Classes

Separate decorator classes were created for each pricing concern: GiftWrapDecorator, DiscountDecorator, and TaxDecorator. Each class overrides only the pricing operations relevant to its responsibility. For example: GiftWrapDecorator adds a fixed £3 surcharge to getPrice(). DiscountDecorator holds -discountPercent and applies a percentage reduction. TaxDecorator holds -vatRate and adds VAT on top of the current price. Because all three implement IProduct through inheritance, they can be stacked in any order: new TaxDecorator(new DiscountDecorator(new GiftWrapDecorator(product))).

DESIGN CHOICES MADE

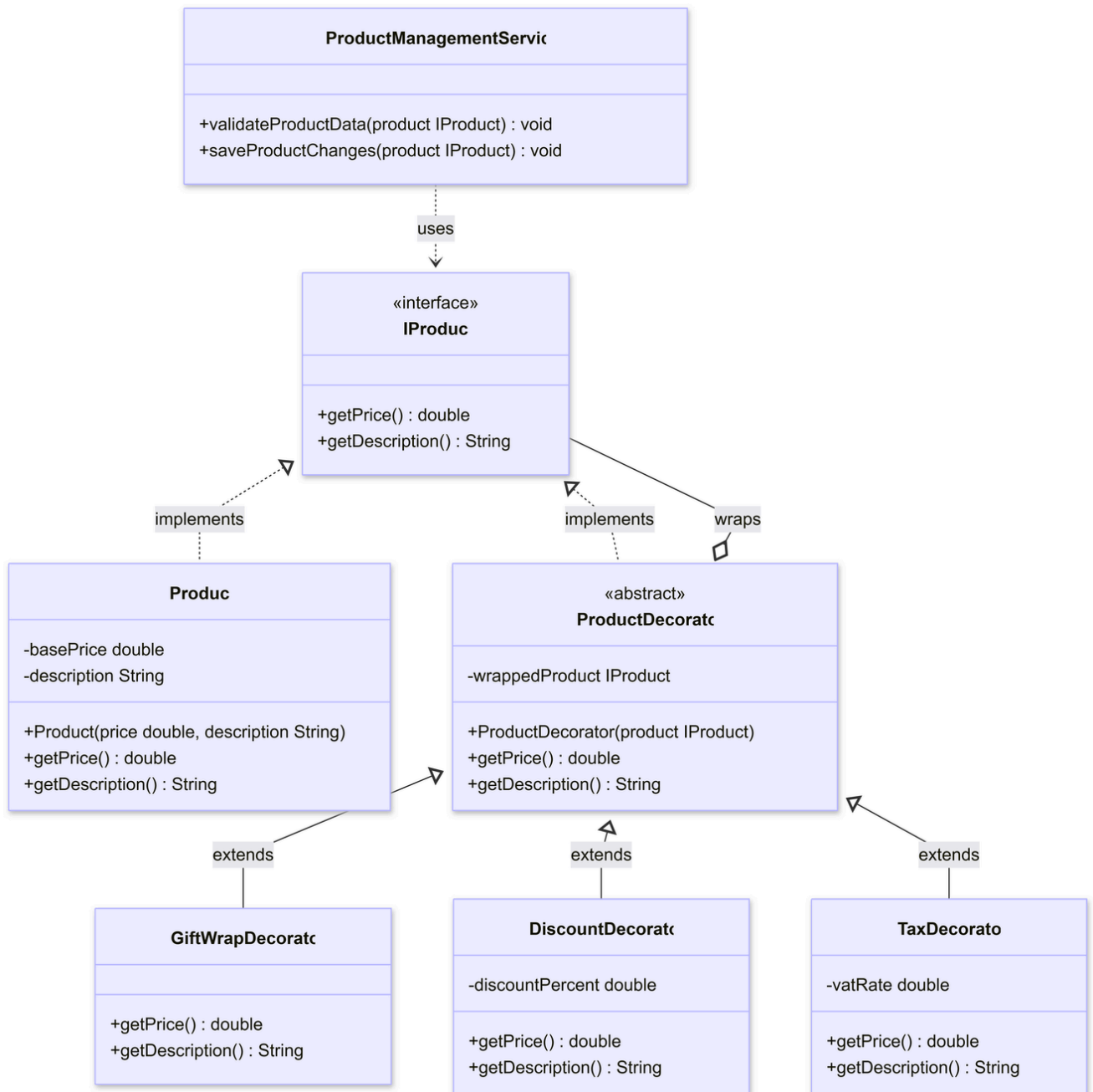
5. ProductManagementService

IntegrationProductManagementService was kept independent of decorator internals. Its methods validateProductData(product: IProduct) and saveProductChanges(product: IProduct) accept the IProduct interface. This means the service can validate or persist a product at any decoration stage without any modification – it does not know or care whether the product is wrapped.

BENEFITS OF THIS DESIGN

Using the Decorator Pattern provides several advantages in the system: eliminates combinatorial subclass explosion for pricing combinations; allows pricing modifications to be added, removed, or reordered at runtime; keeps Product and ProductManagementService closed for modification but open for extension; encapsulates each pricing concern in a dedicated, testable class; supports unlimited stacking – any number of decorators can be chained in any sequence; and models real-world pricing workflows (base → discount → tax → gift wrap) more accurately. One possible drawback is that deeply stacked decorators can make debugging harder, as the final price results from multiple layered calls. However, when each decorator remains focused on a single pricing responsibility, the pattern creates a clean and highly extensible pricing architecture.

CLASS DIAGRAM



ADAPTER PATTERN

What the Adapter Pattern Is

The Adapter Pattern is a structural design pattern that allows objects with incompatible interfaces to collaborate by converting one interface into another that the system expects.

Instead of modifying existing classes or external services directly, the pattern introduces an intermediate adapter object that translates requests and responses between components. The pattern helps improve compatibility, reduce coupling, and simplify integration with external systems while preserving existing implementations.

RELEVANCE TO OUR SYSTEM DESIGN

The Adapter Pattern was selected because the e-commerce platform interacts with multiple delivery and shipment providers that may expose tracking information in different formats.

During delivery tracking:

- tracking information must be requested,
- responses must be standardized,
- provider-specific formats must be converted,
- customers must receive consistent tracking updates.

Without an adapter, the system would need separate logic for every logistics provider, creating tightly coupled and difficult-to-maintain code. Because of this variation between external service interfaces, the Adapter Pattern is highly suitable for simplifying logistics integration. Instead of implementing provider-specific conversion logic inside the frontend or tracking module, the system delegates format conversion to dedicated adapter classes.

DESIGN CHOICES MADE

1. TrackingPage as the Client

The TrackingPage class is designed as the client because it represents the component requesting delivery status information. The page doesn't communicate directly with logistics providers, but it only interacts with:

- ITrackingService
- through the operation:
- trackOrder(orderId : int)

This keeps the frontend independent from external provider implementations.

DESIGN CHOICES MADE

2. ITrackingService Interface

An interface named ITrackingService was used to define tracking operations.

This decision was made because:

- it improves abstraction,
- reduces dependency on specific providers,
- allows integration of new logistics services in the future.

The client depends on the interface rather than concrete implementations.

3. TrackingAdapter as the Translator

TrackingAdapter was designed as the main adapter class because it converts external tracking data into a unified system format.

The adapter internally manages:

- request conversion,
- provider communication,
- response translation,
- standardized tracking output.

The adapter hides incompatibilities between systems and exposes only a single tracking interface.

4. Specialized Logistics Provider Classes

Separate provider classes were created for:

- DHLTrackingService
- FedExTrackingService
- LocalCourierService
- Each provider contains only operations related to communication with that specific provider.

For example:

- DHLTrackingService retrieves DHL tracking data.
- FedExTrackingService retrieves FedEx shipment information.
- LocalCourierService retrieves local delivery status.

This design improves modularity and separates external integrations clearly.

BENEFITS OF THIS DESIGN

Using the Adapter Pattern provides several advantages in the system:

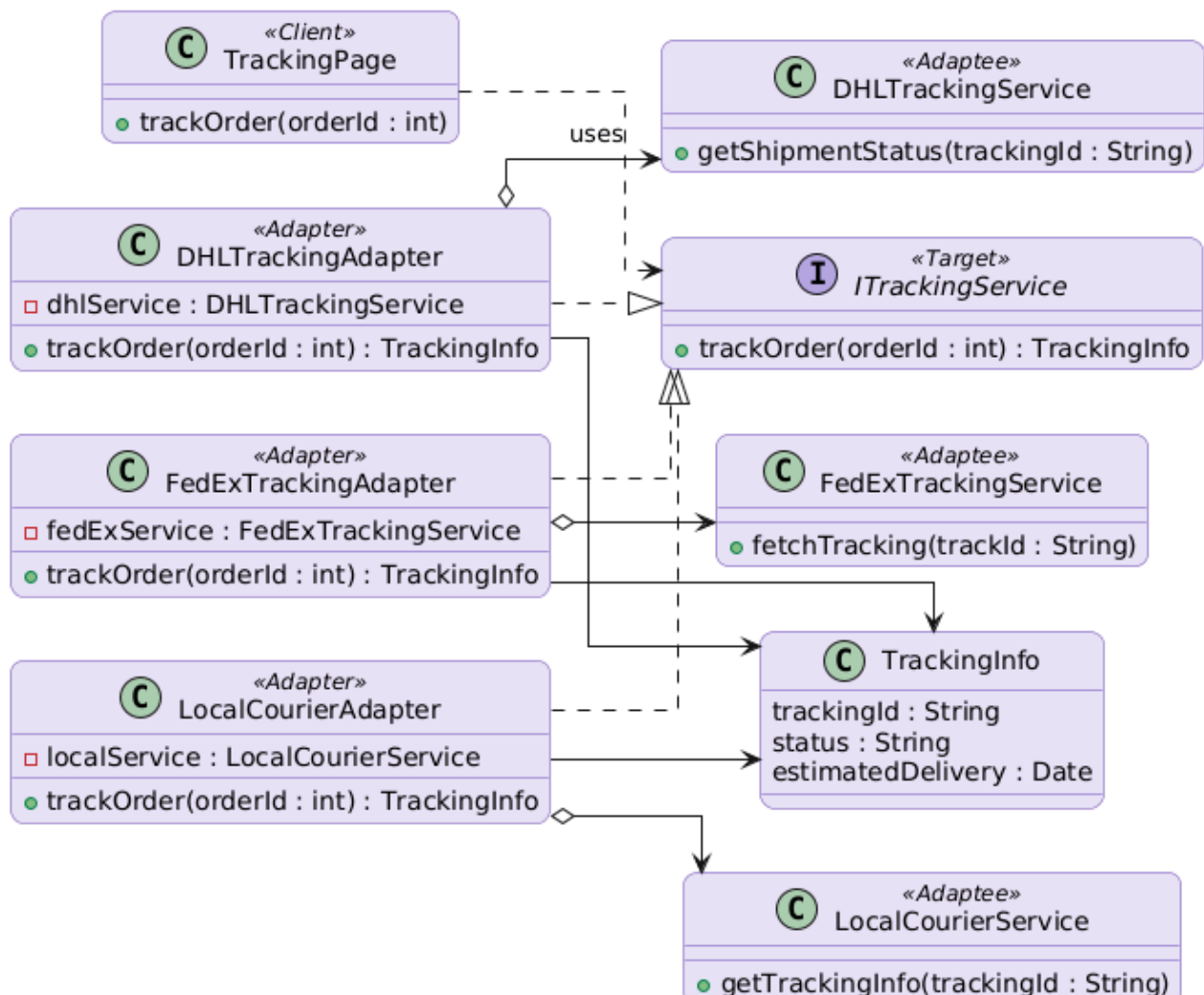
- Reduces dependency on external APIs.
- Simplifies logistics integration.
- Standardizes tracking information.
- Improves maintainability and readability.
- Makes future provider additions easier.
- Hides external implementation differences.

One possible drawback is that additional adapter classes increase system complexity when many providers are introduced.

However, when focused on interface conversion, the Adapter Pattern creates a cleaner and more extensible architecture for delivery tracking.

CLASS DIAGRAM

Adapter Pattern - Delivery Tracking



BUILDER PATTERN

What is the Builder Pattern?

The Builder Pattern is a creational design pattern used to construct complex objects step-by-step.

Instead of creating large objects with complex constructors containing many optional parameters, the Builder Pattern separates the construction process from the final object representation.

The pattern allows the system to:

- create different configurations of an object,
- construct objects gradually,
- and improve readability and maintainability.

The Builder Pattern is especially useful when:

- an object contains many optional attributes,
- different combinations of configurations are possible,
- or object construction becomes too complex for a single constructor.

The pattern improves:

- flexibility,
- scalability,
- readability,
- and separation of construction logic.

RELEVANCE TO OUR SYSTEM DESIGN

The Builder Pattern was selected because the e-commerce platform supports advanced product searching and filtering functionality.

According to the system requirements and use cases, customers can:

- search products,
- filter by category,
- filter by price range,
- filter by availability,
- filter by vendor,
- and sort products using different criteria.

Since every customer may use different search combinations, the search criteria object becomes complex and highly configurable.

RELEVANCE TO OUR SYSTEM DESIGN

For example:

- One customer may search only by category.
- Another customer may filter by category and price range.
- Another customer may apply all available filters and sorting options.

Instead of creating large constructors with many optional parameters, the system gradually constructs a search criteria object using a builder structure.

This design aligns closely with:

- Product Search and Discovery
- Product Filtering
- Product Comparison
- Advanced Product Exploration

DESIGN CHOICES

1. ProductSearchCriteria as the Complex Object

The ProductSearchCriteria class was designed as the final complex object because it stores all filtering and search configuration data selected by the customer.

The class contains configurable search attributes such as:

- category,
- price range,
- vendor,
- availability,
- and sorting preference.

This object represents the final constructed search request used by the product search system.

2. ProductSearchBuilder as the Builder Class

The ProductSearchBuilder class was created to gradually construct ProductSearchCriteria objects.

Instead of passing all filtering parameters directly into a constructor, the builder provides separate methods for configuring each search option individually.

DESIGN CHOICES

This allows:

- flexible object creation,
- cleaner construction logic,
- and optional filter configuration.

The builder contains methods such as:

- `setCategory()`
- `setPriceRange()`
- `setVendor()`
- `setAvailability()`
- `setSortOrder()`
- `build()`

3. ProductDiscoveryService as the Director

The `ProductDiscoveryService` class was selected as the director because it coordinates the search-building process and controls how search criteria objects are constructed.

The service uses `ProductSearchBuilder` to create search configurations depending on customer-selected filters.

This separates:

- object construction logic,
- from the actual product search functionality.

4. Step-by-Step Construction Process

The Builder Pattern was selected because search configurations are created incrementally.

Different customers may provide:

- different filter combinations,
- different sorting preferences,
- and different search configurations.

The builder structure allows the system to construct only the required search properties without forcing unnecessary parameters.

This improves flexibility and avoids constructor complexity.

DESIGN CHOICES

5. Separation of Construction and Representation

The construction logic was separated from the ProductSearchCriteria representation object.

This decision was made because:

- search configuration logic may evolve over time,
- additional filters may be added in future versions,
- and construction behavior should remain isolated from the final object structure.

This improves maintainability and extensibility.

BENEFITS OF THIS DESIGN

Using the Builder Pattern provides several advantages in the system:

- Simplifies creation of complex search objects.
- Avoids constructors with many optional parameters.
- Supports flexible search/filter combinations.
- Improves readability and maintainability.
- Separates object construction from object representation.
- Makes future search extensions easier to implement.
- Supports scalable product discovery functionality.

CLASS DIAGRAM

